

C++ 프로그래밍

열거형 (Enum)

열거형이란?

이름 있는 정수 상수들의 집합을 하나의 타입으로 묶는 것이다.

```
1 // ❌ 매직 넘버 - 0이 뭔지 알 수 없음
2 int direction = 0;
3 if (direction == 0) { ... } // 0이 북쪽? 동쪽?
4
5 // ✅ enum - 의미 있는 이름으로 명확하게
6 enum Direction { NORTH, EAST, SOUTH, WEST };
7 Direction dir = NORTH;
8 if (dir == NORTH) { ... } // 의도가 명확함
```

열거형을 쓰면 **의미 없는 숫자 대신 이름**으로 상태나 종류를 표현할 수 있어 코드의 가독성과 안전성이 높아진다.

C 스타일 enum

선언과 기본값

```
// 첫 번째 값은 0, 이후 1씩 증가
enum Color { RED, GREEN, BLUE };
//           0     1     2

// 값을 직접 지정할 수 있음
enum HttpStatus {
    OK          = 200,
    NOT_FOUND   = 404,
    ERROR       = 500,
};

// 일부만 지정 - 나머지는 이전 값 + 1
enum Priority {
    LOW         = 1,
    MEDIUM,    // 2
    HIGH,       // 3
};
```

사용

```
enum Color { RED, GREEN, BLUE };

Color c = GREEN;

if (c == GREEN) {
    printf("초록색\n");
}

// 정수로 암묵적 변환됨 (주의!)
int n = c;           // n = 1
Color c2 = (Color)2; // BLUE (캐스팅 필요)

// switch와 잘 어울림
switch (c) {
    case RED:    printf("빨강\n"); break;
    case GREEN:  printf("초록\n"); break;
    case BLUE:   printf("파랑\n"); break;
}
```

C 스타일 enum의 문제점

C 스타일 enum은 이름이 전역 네임스페이스에 노출되어 충돌이 발생할 수 있다.

이름 충돌

```
enum Color { RED, GREEN, BLUE };
enum Fruit { APPLE, GREEN, MANGO }; // ❌ GREEN 중복!
```

```
enum Direction { NORTH, EAST };
int NORTH = 10; // ❌ 이미 NORTH 존재!
```

타입 안전성 부재

```
enum Color { RED, GREEN, BLUE };
enum Direction { NORTH, EAST, SOUTH, WEST };
```

```
Color c = RED;
Direction d = NORTH;
```

```
if (c == d) { ... } // ❌ 컴파일은 되지만 논리 오류!
// Color과 Direction을 비교하는 것은 의미 없음
int n = c + d; // ❌ 암묵적 정수 변환
```

해결책 — enum class

```
enum class Color { RED, GREEN, BLUE };
enum class Direction { NORTH, EAST, SOUTH, WEST };
```

```
// ✅ 이름이 enum 안에 한정됨
Color c = Color::GREEN;
Direction d = Direction::NORTH;
```

```
// ✅ 타입이 다른 비교 → 컴파일 오류
// if (c == d) { } // ❌ 컴파일 오류
// int n = c; // ❌ 암묵적 변환 불가
```

```
// ✅ 같은 이름도 충돌 없음
enum class Fruit { APPLE, GREEN, MANGO };
// Color::GREEN 과 Fruit::GREEN 은 완전히 별개
```

enum class (Scoped Enum)

C++11부터 도입된 **타입 안전한** 열거형이다. 새 코드에서는 항상 `enum class` 를 사용하자.

선언과 사용

```
enum class Season {
    SPRING,
    SUMMER,
    FALL,
    WINTER,
};

int main() {
    Season s = Season::SUMMER;

    switch (s) {
        case Season::SPRING: printf("봄\n"); break;
        case Season::SUMMER: printf("여름\n"); break;
        case Season::FALL:   printf("가을\n"); break;
        case Season::WINTER: printf("겨울\n"); break;
    }
}
```

기본 타입 지정

기본 기본 타입은 `int` 다. 필요하면 변경할 수 있다.

```
// 기본 타입을 uint8_t로 지정 (메모리 절약)
enum class Direction : uint8_t {
    NORTH = 0,
    EAST  = 1,
    SOUTH = 2,
    WEST  = 3,
};

// 정수로 변환하려면 명시적 캐스팅 필요
Direction d = Direction::EAST;
int n = static_cast<int>(d);    // n = 1
printf("방향 번호: %d\n", n);
```

`static_cast<int>()` — C++ 권장 캐스팅 방식

enum class 활용 예

상태 머신 (State Machine)

```
enum class TrafficLight { RED, YELLOW, GREEN };

TrafficLight next(TrafficLight current) {
    switch (current) {
        case TrafficLight::RED:    return TrafficLight::GREEN;
        case TrafficLight::GREEN:  return TrafficLight::YELLOW;
        case TrafficLight::YELLOW: return TrafficLight::RED;
    }
    return current; // unreachable
}

int main() {
    TrafficLight light = TrafficLight::RED;
    light = next(light); // GREEN
    light = next(light); // YELLOW
    light = next(light); // RED
}
```

비트 플래그 (Bit Flag)

```
enum class Permission : unsigned int {
    NONE    = 0,
    READ    = 1 << 0, // 0b001 = 1
    WRITE   = 1 << 1, // 0b010 = 2
    EXECUTE = 1 << 2, // 0b100 = 4
};

// 비트 연산으로 여러 권한 조합
unsigned int perm =
    static_cast<unsigned int>(Permission::READ) |
    static_cast<unsigned int>(Permission::WRITE);

// 특정 권한 확인
bool can_read = perm &
    static_cast<unsigned int>(Permission::READ); // true
```

요약

```
// C 스타일 - 이름이 전역에 노출, 암묵적 정수 변환 가능
enum Color { RED, GREEN, BLUE };
Color c = GREEN;           // 이름 그대로 사용
int n = c;                 // 암묵적 변환 허용

// C++ 스타일 (권장) - 이름이 enum 안에 한정, 타입 안전
enum class Color { RED, GREEN, BLUE };
Color c = Color::GREEN;    // 반드시 Color:: 접두사
int n = static_cast<int>(c); // 명시적 변환만 허용
```

항목	enum (C 스타일)	enum class (C++11)
이름 범위	전역 노출	열거형 내부로 한정
이름 충돌	발생 가능	없음
정수 변환	자동 (암묵적)	명시적 캐스팅 필요
타입 비교	다른 enum끼리 비교 가능	컴파일 오류
기반 타입 지정	불가	가능 (: uint8_t 등)

새 코드에서는 항상 `enum class` 를 사용하자.